UNIVERSIDADE ESTADUAL DO PIAUÍ – UESPI

Campus Prof. Alexandre Alves de Oliveira

Curso de Tecnologia em Sistemas de Computação

Disciplina: Tópicos em Banco de Dados

Professor: Fco Rocha

Sistema de Transportadora

Modelagem de Banco de Dados Relacional e NoSQL

Grupo 6 – Transportadora

INTEGRANTES

Felipe Martins

Vitor Santos

Francisco Osmar

Gabriel Lima

Parnaíba – PI

2026

1 Requisitos e decisões de modelagem

Este documento modela o banco de dados de uma transportadora a partir dos requisitos da Seção 3 do enunciado. Foi elaborado por Felipe Martins, Vitor Santos, Francisco Osmar e Gabriel Lima (Grupo 6). A Tarefa 1 apresenta o modelo entidade-relacionamento (MER) e o diagrama entidade-relacionamento (DER); a Tarefa 2 mapeia esse modelo para o modelo NoSQL de documentos. A Seção 1.3 registra as premissas adotadas para os pontos que o enunciado deixa em aberto, e a Seção 1.2 descreve os dois requisitos adicionais criados pelo grupo.

1.1 Cenário

O processo começa quando um cliente (remetente) contrata a transportadora e informa o endereço da coleta. A transportadora registra a coleta com os dados do remetente, do destinatário (que também é cliente) e da mercadoria. As mercadorias são levadas ao depósito da cidade, separadas por cidade ou região de destino e carregadas em caminhões. Para cada mercadoria é emitida uma nota fiscal, e o valor do frete é cobrado por meio de uma fatura. O modelo descrito a seguir cobre esse fluxo e inclui os requisitos adicionais RA1 e RA2.

1.2 Requisitos adicionais

O enunciado exige dois requisitos adicionais que provoquem uma nova entidade, relação, agregação ou hierarquia. Foram definidos os seguintes.

RA1 — Hierarquia de tipos de encomenda. A transportadora precisa tratar quatro tipos de encomenda, cada um com informações e regras próprias:

- **comum**: sem exigências específicas;
- **frágil**: cuidados especiais, empilhamento máximo e necessidade de manter em pé;
- **perecível**: faixa de temperatura permitida e validade;
- **perigosa**: número ONU, classe de risco e grupo de embalagem.

Esse requisito gera uma **hierarquia de especialização** da entidade `Encomenda`, com quatro subtipos. A especialização é total (toda encomenda é de um dos quatro tipos) e disjunta (uma encomenda pertence a um único tipo). Não se trata de um simples atributo novo, pois cada subtipo possui atributos e restrições próprios.

RA2 — Embarque e ocorrências. Uma nota fiscal pode ser transportada em um ou mais trechos, cada um realizado por um caminhão e com uma cidade de destino. Durante o trecho podem ocorrer eventos, como atraso, avaria, extravio ou entrega. O registro da ocorrência depende do conjunto *nota + caminhão + trecho*, e não apenas da nota. Esse

requisito gera uma **agregação**: o relacionamento `EMBARCA` entre `NotaFiscal` e `Caminhao` é tratado como um todo, materializado pela entidade `Embarque`; as ocorrências são registradas nesse contexto, por meio da entidade `Ocorrencia`.

1.3 Premissas adotadas

O enunciado não define alguns pontos necessários à modelagem. As premissas abaixo foram adotadas e são usadas de forma consistente no restante do trabalho.

P1. Todos os clientes são pessoas físicas, identificadas por CPF. O destinatário é um cliente previamente cadastrado.

P2. Uma coleta pode conter uma ou mais encomendas; cada encomenda pertence a exatamente uma coleta.

P3. Cada encomenda gera exatamente uma nota fiscal, e cada nota se refere a exatamente uma encomenda.

P4. Cada nota é incluída em exatamente uma fatura; uma fatura pode agrupar várias notas. O pagador da fatura é o remetente ou o destinatário.

P5. Existe no máximo um depósito por cidade, e nem toda cidade possui depósito. Cada coleta é entregue a um depósito.

P6. Um caminhão atende uma ou mais cidades, e uma cidade pode ser atendida por vários caminhões.

P7. Uma nota pode ter um ou mais embarques (trechos). Cada embarque usa um caminhão e tem uma cidade de destino; o último trecho coincide com a cidade de destino da coleta.

P8. O valor do frete é 5% do valor total da nota. O percentual é armazenado para permitir exceções.

P9. A "data de entrega da coleta" é a data em que a coleta é entregue ao depósito; a "data de entrega da mercadoria" é a data registrada na nota fiscal.

P10. A região agrupa cidades e é usada para separar as mercadorias no depósito.

P11. "Endereço completo" corresponde a logradouro, número, complemento, bairro, cidade, UF e CEP.

2 Tarefa 1 — Modelagem com MER/DER

2.1 Entidades e atributos

A Tabela 1 apresenta as entidades do modelo, seus atributos e a chave primária. As chaves estrangeiras estão indicadas por "(FK)" e o atributo derivado por "(derivado)". Para manter

o DER legível, os atributos são detalhados na tabela e não repetidos na visão geral da Figura 1.

Tabela 1: Entidades, atributos e chaves

Entidade	Atributos	Chave primária
Cliente	cpf; nome_completo; telefone; email; endereco (logradouro, número, complemento, bairro, cidade, UF, CEP).	cpf
Coleta	codigo_coleta; data_coleta; data_entrega_coleta; cpf_remetente (FK); cpf_destinatario (FK); nome_remetente; endereco_coleta; telefone_remetente; nome_destinatario; endereco_entrega; telefone_destinatario; codigo_deposito_origem (FK); codigo_cidade_destino (FK).	codigo_coleta
Encomenda	codigo_encomenda; codigo_coleta (FK); descricao; valor; peso_kg; volume_m3; tipo_encomenda (discriminador).	codigo_encomenda
EncomendaComum	codigo_encomenda (PK/FK). Sem atributos específicos.	codigo_encomenda
EncomendaFragil	codigo_encomenda (PK/FK); cuidados_especiais; empilhamento_maximo; manter_em_pe.	codigo_encomenda
EncomendaPerecivel	codigo_encomenda (PK/FK); temperatura_min_c; temperatura_max_c; validade.	codigo_encomenda
EncomendaPerigosa	codigo_encomenda (PK/FK); numero_onu; classe_risco; grupo_embalagem.	codigo_encomenda
NotaFiscal	numero_nota; codigo_encomenda (FK, único); codigo_fatura (FK); data_emissao; valor_total; percentual_frete; valor_frete (derivado); data_entrega_destinatario.	numero_nota
Fatura	numero_fatura; cpf_pagador (FK); data_emissao; data_vencimento; valor_frete_total (derivado).	numero_fatura
Caminhao	placa; modelo; capacidade_kg; capacidade_m3.	placa
Cidade	codigo_cidade; nome; uf; codigo_regiao (FK).	codigo_cidade
Regiao	codigo_regiao; nome.	codigo_regiao
Deposito	codigo_deposito; nome; endereco; codigo_cidade (FK, único).	codigo_deposito
Embarque	codigo_embarque; numero_nota (FK); placa (FK); codigo_cidade_destino (FK); ordem_trecho; data_hora_embarque; data_hora_desembarque.	codigo_embarque
Ocorrencia	codigo_ocorrencia; codigo_embarque (FK); tipo; data_hora; descricao; status.	codigo_ocorrencia

Os campos `nome_remetente`, `endereco_coleta`, `telefone_remetente`, `nome_destinatario`, `endereco_entrega` e `telefone_destinatario` são cópias históricas dos dados do cliente no momento da coleta, exigidas pelo documento de coleta.

2.2 Relacionamentos e cardinalidades

A Tabela 2 resume os relacionamentos, as cardinalidades e a participação. A notação (m, n) indica o mínimo e o máximo de ocorrências de uma entidade associadas a uma ocorrência da outra.

Tabela 2: Relacionamentos, cardinalidades e participação

Relacionamento	Entidades	Cardinalidade	Regra de negócio (participação)
`ENVIA`	Cliente–Coleta	1:N	Total em Coleta: um cliente pode ser remetente de várias coletas; cada coleta tem um remetente.
`RECEBE`	Cliente–Coleta	1:N	Total em Coleta: um cliente pode ser destinatário de várias coletas; cada coleta tem um destinatário.
`CONTEM`	Coleta–Encomenda	1:N	Total em Encomenda: uma coleta contém uma ou mais encomendas; cada encomenda pertence a uma coleta.
`GERA`	Encomenda–NotaFiscal	1:1	Total nos dois lados: cada encomenda gera uma nota; cada nota se refere a uma encomenda.
`COBRA`	NotaFiscal–Fatura	N:1	Total em NotaFiscal: uma fatura agrupa várias notas; cada nota pertence a uma fatura.
`PAGA`	Cliente–Fatura	1:N	Total em Fatura: o pagador é o remetente ou o destinatário; um cliente pode pagar várias faturas.
`ORIGINA`	Deposito–Coleta	1:N	Total em Coleta: cada coleta é entregue a um depósito; um depósito recebe várias coletas.
`ENTREGA_EM`	Coleta–Cidade	N:1	Total em Coleta: cada coleta tem uma cidade de destino; uma cidade recebe várias coletas.

Relacionamento	Entidades	Cardinalidade	Regra de negócio (participação)
SEDIA	Deposito–Cidade	1:1	Total em Deposito e parcial em Cidade: no máximo um depósito por cidade; todo depósito fica em uma cidade.
AGRUPIA	Cidade–Regiao	N:1	Total em Cidade: cada cidade pertence a uma região; uma região agrupa várias cidades.
ATENDE	Caminhao–Cidade	N:N	Total em Caminhao, parcial em Cidade: um caminhão atende uma ou mais cidades (P6); uma cidade pode não ter caminhões cadastrados.
EMBARCA	NotaFiscal–Caminhao	N:N	Total em NotaFiscal, parcial em Caminhao: uma nota tem um ou mais embarques (P7); um caminhão pode não ter embarques.
DESTINO_TRECHO	Embarque–Cidade	N:1	Total em Embarque: cada trecho tem uma cidade de destino.
REGISTRA	Embarque–Ocorrencia	1:N	Total em Ocorrencia: um embarque pode ter várias ocorrências; cada ocorrência pertence a um embarque.

2.3 Diagramas

A Figura 1 apresenta a visão geral do DER. Para evitar poluição visual, os atributos foram omitidos dessa figura; eles estão na Tabela 1 e nos detalhes das Figuras 2 e 3. Todas as entidades, relacionamentos e cardinalidades do modelo estão representados.

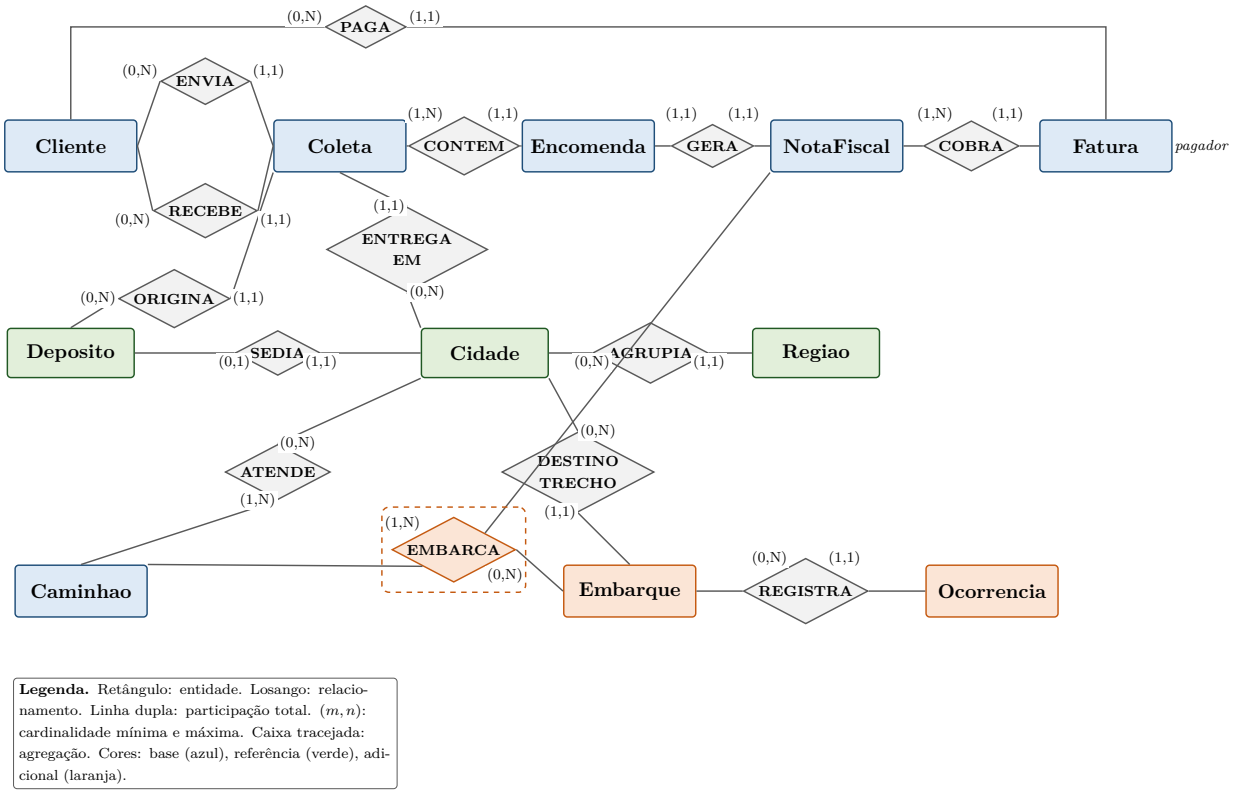


Figura 1: Visão geral do DER do Sistema de Transportadora

A Figura 2 detalha o requisito adicional RA1. A especialização é representada pelo círculo com a letra "d", que indica especialização disjunta; a linha dupla entre o supertipo e o círculo indica participação total. A Figura 3 detalha o requisito adicional RA2: o losango `EMBARCA` é envolvido pela caixa tracejada (agregação) e materializado pela entidade `Embarque`, que por sua vez se relaciona com `Ocorrencia`.

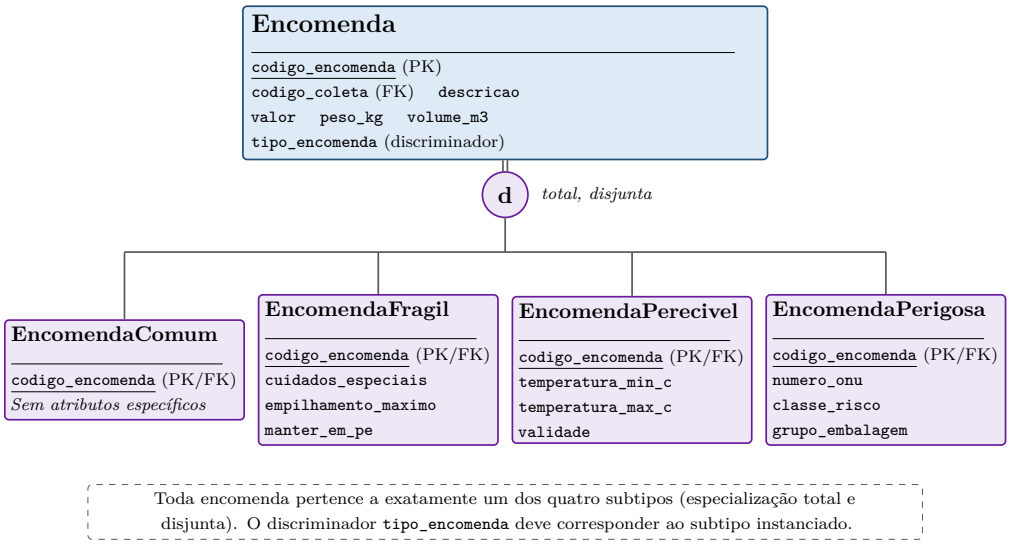


Figura 2: Hierarquia de especialização de `Encomenda` (RA1)

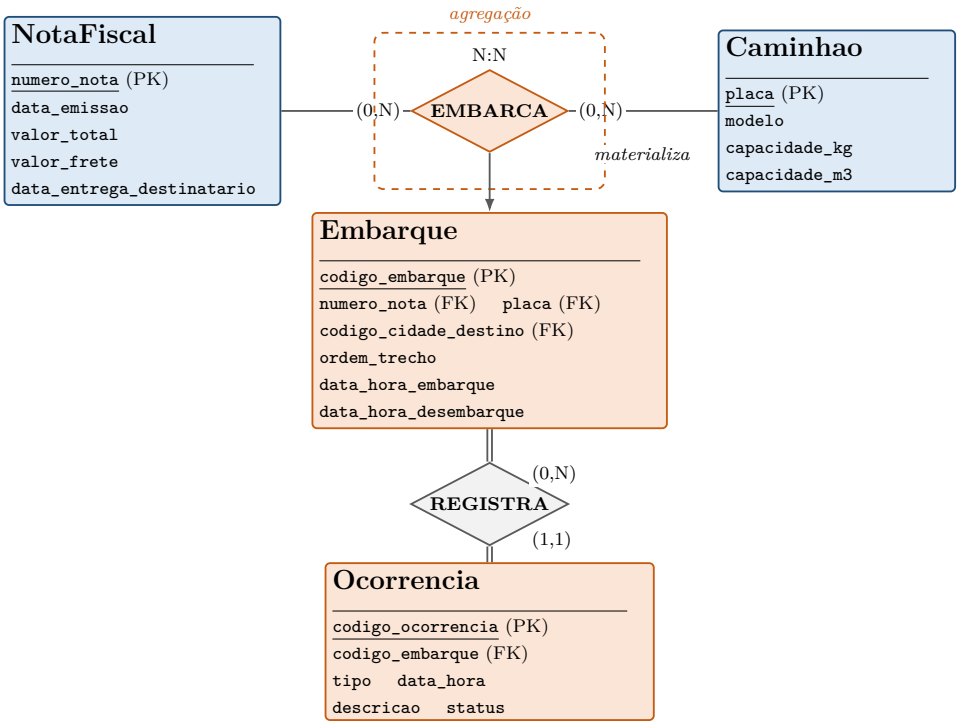


Figura 3: Agregação `EMBARCA` e entidades `Embarque` e `Ocorrencia` (RA2)

2.4 Como os requisitos da Seção 3 foram modelados

A Tabela 3 relaciona cada item do enunciado à decisão de modelagem correspondente. Os requisitos adicionais RA1 e RA2 estão descritos na Seção 1.2 e representados nas Figuras 2 e 3.

Tabela 3: Correspondência entre os requisitos da Seção 3 e o MER

Item	Requisito	Como foi modelado no MER
R01	O cliente contata a transportadora e é registrada uma coleta no endereço indicado.	Entidade `Coleta`, com `data_coleta` e `endereco_coleta`, relacionada ao cliente remetente por `ENVIA`.
R02	O cliente é o remetente e o destinatário também é um cliente.	Dois relacionamentos 1:N entre `Cliente` e `Coleta`: `ENVIA` (remetente) e `RECEBE` (destinatário).
R03	Guardar nome completo, CPF, telefone, e-mail e endereço completo do remetente e do destinatário.	Atributos em `Cliente`, com `cpf` como chave primária. O endereço é um atributo composto.

Item	Requisito	Como foi modelado no MER
R04	O documento de coleta registra CPF, nome, endereço e telefone de remetente e destinatário.	`Coleta` guarda as FKs `cpf_remetente` e `cpf_destinatario` e cópias históricas de nome, endereço e telefone de ambos.
R05	Dados da mercadoria: descrição, valor, peso e volume em metros cúbicos.	Entidade `Encomenda`, com `descricao`, `valor`, `peso_kg` e `volume_m3`, em relacionamento 1:N com `Coleta` (`CONTEM`).
R06	Emitir nota fiscal com os dados do remetente e do destinatário, buscando os dados da coleta, inclusive a mercadoria.	`NotaFiscal` em 1:1 com `Encomenda` (`GERA`); os dados de remetente e destinatário são obtidos pelo caminho `NotaFiscal`→`Encomenda`→`Coleta`→`Cliente`, sem duplicação no MER.
R07	Os caminhões que fazem coletas em uma cidade chegam ao depósito; há um depósito por cidade; as mercadorias são agrupadas por cidade ou região de destino.	`Deposito` em 1:1 com `Cidade` (`SEDIA`); `Coleta` se relaciona com `Deposito` (`ORIGINA`) e com a cidade de destino (`ENTREGA_EM`); `Cidade` se relaciona com `Regiao` (`AGRUPIA`).
R08	Para cada mercadoria é gerada uma nota fiscal.	Relacionamento 1:1 `GERA`, com participação total nos dois lados.
R09	A coleta tem uma data de entrega.	Atributo `data_entrega_coleta` em `Coleta` (entrega ao depósito, conforme a premissa P9).
R10	As mercadorias são carregadas em caminhões; cada caminhão vai para uma ou muitas cidades; mais de um caminhão pode ir para a mesma cidade.	Relacionamento N:N `ATENDE` entre `Caminhao` e `Cidade`; o transporte efetivo é registrado por `EMBARCA`, entre `NotaFiscal` e `Caminhao`, materializado por `Embarque`.
R11	Calcular o valor do frete, normalmente 5% do valor total da nota; a nota tem data de entrega ao destinatário.	Atributos `valor_total`, `percentual_frete`, `valor_frete` (derivado) e `data_entrega_destinatario` em `NotaFiscal`.

Item	Requisito	Como foi modelado no MER
R12	A nota não é paga pelo cliente; ao gerar a nota é gerada uma fatura do frete, com número, datas de emissão e vencimento, valor e nome do pagador; a fatura pode envolver várias notas.	`Fatura` em 1:N com `NotaFiscal` (`COBRA`); `Cliente` se relaciona com `Fatura` no papel de pagador (`PAGA`); `valor_frete_total` é derivado das notas.

3 Tarefa 2 — Mapeamento para NoSQL

3.1 Estratégia geral

O modelo foi mapeado para o MongoDB, que organiza os dados em coleções de documentos BSON. As decisões de mapeamento seguiram quatro critérios:

1. **Embutir** dados que são acessados juntos, que dependem do documento pai e cuja quantidade é limitada (encomendas de uma coleta, trechos e ocorrências de uma nota).
2. **Referenciar** entidades com vida própria, compartilhadas por muitos documentos ou com crescimento não limitado (clientes, caminhões, cidades, regiões e depósitos).
3. **Guardar snapshots** dos dados usados em documentos fiscais (remetente, destinatário, encomenda, cidade e região), para preservar o histórico mesmo que o cadastro seja alterado.
4. **Desnormalizar** apenas dados pequenos e estáveis, como o nome da cidade e da região, para evitar junções em tempo de leitura.

O resultado são oito coleções: `clientes`, `coletas`, `notas_fiscais`, `faturas`, `caminhoes`, `cidades`, `regioes` e `depositos`.

3.2 Entidades

A Tabela 4 comenta o mapeamento de cada entidade. As entidades dependentes (`Encomenda`, `Embarque` e `Ocorrencia`) não geram coleções próprias: elas são embutidas nos documentos dos quais dependem.

Tabela 4: Mapeamento das entidades para o MongoDB

Entidade	Onde fica	Estratégia e justificativa
`Cliente`	Coleção `clientes`; `_id` = CPF.	Referência. O cliente é compartilhado por coletas e faturas e tem vida própria. O endereço é um subdocumento.
`Coleta`	Coleção `coletas`; `_id` = `codigo_coleta`.	Raiz de agregado. Contém snapshots de remetente e destinatário, referência ao depósito e à cidade de destino e o array de encomendas.
`Encomenda`	Array `encomendas[]` em `coletas`.	Embutir. A encomenda não existe sem a coleta e é sempre consultada com ela; a atualização do conjunto é atômica.
`Encomenda-Comum`, `Fragil`, `Perecivel`, `Perigosa`	Campo `tipo` + subdocumento `detalhes` em cada item de `encomendas[]`.	Polimorfismo. Os subtipos compartilham os atributos comuns; evita quatro coleções e mantém a leitura do agregado em uma única consulta.
`NotaFiscal`	Coleção `notas_fiscais`; `_id` = `numero_nota`.	Raiz de agregado. Guarda snapshots de remetente, destinatário, encomenda, cidade e região, além dos transportes e ocorrências.
`Fatura`	Coleção `faturas`; `_id` = `numero_fatura`.	Raiz de agregado. Contém o snapshot do pagador e um resumo das notas (número e valor do frete), sem repetir a nota completa.
`Caminhao`	Coleção `caminhoes`; `_id` = `placa`.	Referência. É consultado isoladamente e possui um array de referências às cidades atendidas.
`Cidade`	Coleção `cidades`; `_id` = `codigo_cidade`.	Referência. Guarda o nome da região desnormalizado, pois é um dado pequeno e estável.
`Regiao`	Coleção `regioes`; `_id` = `codigo_regiao`.	Referência. É compartilhada por várias cidades e usada para agrupar mercadorias.
`Deposito`	Coleção `depositos`; `_id` = `codigo_deposito`.	Referência. É compartilhado por muitas coletas; `cidade_id` recebe índice único para garantir um depósito por cidade.
`Embarque`	Array `transportes[]` em `notas_fiscais`.	Embutir. Depende da nota, o número de trechos é limitado e o acesso é feito junto com a nota. O caminhão é referenciado.
`Ocorrencia`	Array `transportes[].ocorrencias[]`.	Embutir. Depende do embarque; o array preserva o contexto do trecho (nota, caminhão e cidade).

3.3 Relacionamentos

A Tabela 5 mostra como cada relacionamento do MER foi representado. Em geral, os relacionamentos 1:N são resolvidos por embutimento quando o lado "N" é dependente e limitado, e por referência quando o lado "N" é uma entidade compartilhada.

Tabela 5: Mapeamento dos relacionamentos para o MongoDB

Relacionamento	Representação no documento	Estratégia e justificativa
ENVIA	`coletas.remetente` (`cpf`, nome, endereço, telefone).	Snapshot + referência implícita ao CPF. O documento de coleta precisa dos dados como foram registrados.
RECEBE	`coletas.destinatario` (`cpf`, nome, endereço, telefone).	Snapshot + referência implícita ao CPF, pelo mesmo motivo.
CONTEM	`coletas.encomendas[]`.	Embutir. As encomendas são dependentes da coleta e acessadas junto com ela.
GERA	`notas_fiscais.encomenda_id` (único) + snapshot em `notas_fiscais.encomenda`.	Referência única para garantir o 1:1; o snapshot preserva os dados da mercadoria na emissão.
COBRA	`notas_fiscais.fatura_id`; resumo em `faturas.notas[]`.	Referência da nota para a fatura e array-resumo na fatura, evitando uma segunda consulta para listar as notas.
PAGA	`faturas.pagador` (`cpf`, nome).	Snapshot do cliente pagador; o CPF identifica o cliente na coleção `clientes`.
ORIGINA	`coletas.deposito_origem` (`codigo`, nome).	Referência com snapshot do nome; o depósito é compartilhado e tem vida própria.
ENTREGA_EM	`coletas.cidade_destino` (`codigo`, nome, UF, região).	Referência com desnormalização. A cidade e a região são pequenas e estáveis, e são necessárias na leitura da coleta.
SEDIA	`depositos.cidade_id` (único) + `cidades` sem array de depósitos.	Referência no lado do depósito; o índice único garante a cardinalidade 1:1.
AGRUPIA	`cidades.regiao` (`codigo`, nome).	Referência com desnormalização do nome da região.

Relacionamento	Representação no documento	Estratégia e justificativa
ATENDE	caminhoes.cidades_atendidas[] com codigo, nome e UF.	Array de referências. A lista é limitada e consultada com o caminhão; índice multikey para buscar por cidade.
EMBARCA	notas_fiscais.transportes[] com caminhao_id, cidade de destino, ordem e datas.	Embutir + referência. O embarque depende da nota; o caminhão é referenciado e indexado.
DESTINO_TRECHO	notas_fiscais.transportes[].cidade_destino.	Referência com snapshot da cidade de cada trecho.
REGISTRA	notas_fiscais.transportes[].ocorrencias[].	Embutir. A ocorrência depende do embarque; o array mantém o contexto do trecho.

3.4 Hierarquia (RA1)

A hierarquia de Encomenda foi mapeada como **polimorfismo em um único array**. Cada item de coletas.encomendas[] possui o campo tipo, que funciona como discriminador, e um subdocumento detalhes cujo conteúdo varia conforme o tipo. A alternativa de criar quatro coleções foi descartada porque os subtipos compartilham a maior parte dos atributos e são quase sempre lidos junto com a coleta; a validação dos campos obrigatórios de cada tipo pode ser feita por um *JSON Schema* da coleção. O trecho a seguir mostra uma encomenda frágil e uma perecível.

```
"encomendas": [
  { "codigo": "E001", "tipo": "fragil", "descricao": "Vidros decorativos",
    "valor": 1200.00, "peso_kg": 8.5, "volume_m3": 0.12,
    "detalhes": { "cuidados_especiais": "Nao empilhar",
                  "empilhamento_maximo": 2, "manter_em_pe": true } },
  { "codigo": "E002", "tipo": "perecivel", "descricao": "Iogurte natural",
    "valor": 350.00, "peso_kg": 12.0, "volume_m3": 0.08,
    "detalhes": { "temperatura_min_c": 2, "temperatura_max_c": 8,
                  "validade": "2026-11-05" } }
]
```

Listing 1: Encomendas frágil e perecível embutidas em uma coleta

3.5 Agregação (RA2)

A agregação EMBARCA foi mapeada como um array de subdocumentos dentro da nota fiscal. Cada elemento de notas_fiscais.transportes[] representa um embarque (trecho), referencia o caminhão por caminhao_id e contém o array ocorrencias[]. Assim, a ocorrência fica registrada no contexto correto (nota, caminhão e trecho), sem precisar de uma coleção

independente e sem risco de ficar órfã. O trecho abaixo ilustra uma nota com dois trechos e uma ocorrência.

```
"transportes": [
  { "codigo_embarque": "EM001", "caminhao_id": "ABC1D23",
    "cidade_destino": { "codigo": "PIR", "nome": "Piripiri", "uf": "PI" },
    "ordem_trecho": 1,
    "data_hora_embarque": "2026-10-20T08:30:00Z",
    "data_hora_desembarque": "2026-10-20T13:10:00Z",
    "ocorrencias": [
      { "codigo": "OC01", "tipo": "atraso",
        "data_hora": "2026-10-20T13:40:00Z",
        "descricao": "Trecho liberado com 30 min de atraso",
        "status": "resolvida" } ] },
  { "codigo_embarque": "EM002", "caminhao_id": "XYZ9K88",
    "cidade_destino": { "codigo": "THE", "nome": "Teresina", "uf": "PI" },
    "ordem_trecho": 2,
    "data_hora_embarque": "2026-10-20T14:00:00Z",
    "data_hora_desembarque": "2026-10-20T18:30:00Z",
    "ocorrencias": [] }
]
```

Listing 2: Nota fiscal com embarques e ocorrências aninhados

3.6 Índices e padrões de acesso

A Tabela 6 lista os índices que sustentam as consultas mais frequentes do sistema. Além das chaves primárias (`_id`), foram criados índices simples, únicos e multikey.

Tabela 6: Índices previstos no modelo NoSQL

Coleção	Índice	Finalidade
`coletas`	`{"destinatario.cpf": 1}`	Histórico de coletas por destinatário.
`coletas`	`{"cidade_destino.codigo": 1}`	Coletas por cidade de destino.
`notas_fiscais`	`{encomenda_id: 1}`, único	Garantir 1:1 NotaFiscal–Encomenda (impede notas duplicadas para a mesma encomenda).
`notas_fiscais`	`{fatura_id: 1}`	Recuperar as notas de uma fatura.
`notas_fiscais`	`{"transportes.caminhao_id": 1}`	Histórico de notas por caminhão (multikey).
`notas_fiscais`	`{"transportes.ocorrencias.tipo": 1}`	Consultar ocorrências por tipo (multikey).
`faturas`	`{"pagador.cpf": 1}`	Histórico de faturas por pagador.
`caminhoes`	`{"cidades_atendidas.codigo": 1}`	Caminhões que atendem uma cidade (multikey).

3.7 Garantias de integridade no MongoDB

O modelo relacional usa chaves estrangeiras e restrições declarativas para assegurar integridade referencial. No MongoDB, essas garantias precisam ser tratadas de forma diferente:

- **Índice único em `notas_fiscais.encomenda_id`** impede que duas notas fiscais referenciem a mesma encomenda (garante o lado "1" do 1:1). Para assegurar que *toda* encomenda tenha uma nota fiscal, a aplicação deve criar a nota no mesmo fluxo da encomenda (transação ou lógica atômica na escrita).
- **Consistência NotaFiscal–Fatura**: a nota armazena `fatura_id` (referência) e a fatura mantém um array-resumo `notas[]` (desnormalização controlada). A sincronização é responsabilidade da aplicação: ao criar uma nota, a fatura é atualizada na mesma operação (transação MongoDB multi-documento). Ao cancelar uma nota, a fatura deve ser ajustada.
- **Integridade de Embarque–Ocorrência**: ao embutir `ocorrencias[]` dentro de `transportes[]`, a integridade é estrutural — não existe ocorrência sem embarque pai.
- **Cardinalidade ATENDE (Caminhao 1:N)**: o array `cidades_atendidas[]` não impede um caminhão sem cidades. A validação de mínimo 1 (premissa P6) deve ser feita na aplicação antes de persistir.
- **Transações multi-documento**: operações que afetam `notas_fiscais` e `faturas` simultaneamente (criar nota + atualizar fatura, cancelar nota + ajustar fatura) devem usar `session.startTransaction()` do MongoDB para atomicidade.
- **JSON Schema**: validações de estrutura (campos obrigatórios, tipos, enum para `tipo_encomenda`, `tipo_ocorrencia`) são definidas via comando `db.createCollection` com validador `{$jsonSchema:...}`.

3.8 Comentários sobre as escolhas

O modelo embute apenas os dados dependentes e limitados, referencia as entidades compartilhadas e usa snapshots para preservar o histórico dos documentos fiscais. Os valores `valor_frete` e `valor_frete_total` são derivados: podem ser calculados na escrita ou em uma *view*, e a aplicação mantém a consistência ao criar ou alterar notas e faturas.

4 Considerações finais

O modelo apresentado cobre os requisitos da Seção 3 do enunciado: clientes nos papéis de remetente, destinatário e pagador; coletas com encomendas; depósitos por cidade; agrupamento por região; notas fiscais com frete; faturas com várias notas; e caminhões que

atendem cidades. Os dois requisitos adicionais ampliam o modelo sem alterar o núcleo: o RA1 introduz a hierarquia de tipos de encomenda e o RA2 introduz o embarque com ocorrências, que dá origem à agregação.

Para o NoSQL, a separação em oito coleções manteve as entidades independentes como referências e embutiu apenas os dados dependentes e limitados. Os snapshots garantem que notas e faturas preservem a informação histórica, e a desnormalização de cidade e região reduz consultas sem criar duplicação relevante. As escolhas priorizam a leitura do agregado e a simplicidade de manutenção, que são os padrões de acesso predominantes no problema.

Referências

[1] ELMASRI, R.; NAVATHE, S. B. *Sistemas de banco de dados*. 7. ed. São Paulo: Pearson, 2019.

[2] HEUSER, C. A. *Projeto de banco de dados*. 6. ed. Porto Alegre: Bookman, 2009.

[3] MONGODB. *MongoDB Manual: Data Modeling*. Disponível em: https://www.mongodb.com/docs/manual/data-modeling/. Acesso em: 27 out. 2026.